

AI-powered bioinformatic analysis using R, #4

Data Wrangling with tidyverse

Wei-Hua Chen (CC BY-NC 4.0)

HUST, China

2026-07-16

Table of contents I

- 1 The Pipe Operator
- 2 dplyr Core Verbs
- 3 tidyr — Reshaping Data
- 4 Joining Tables
- 5 Advanced dplyr
- 6 Summary & Exercises



Section 1

The Pipe Operator

What is the Pipe?

The pipe `%>%` comes from the **magrittr** package by Stefan Milton Bache. It passes the result of the left-hand side as the **first argument** to the right-hand side function.

Traditional code (nested, hard to read):

```
# Step-by-step, storing intermediate results
a <- subset(swiss, Fertility > 20)
cor.test(a$Fertility, a$Education)
```

Pearson's product-moment correlation

```
data:  a$Fertility and a$Education
t = -5.9536, df = 45, p-value = 3.659e-07
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.7987075 -0.4653206
sample estimates:
      cor
-0.6637889
```

What is the Pipe?, cont.

Pipe version (left-to-right, easy to read):

```
library(tidyverse)
library(magrittr)

swiss %>%
  subset(., Fertility > 20) %$%
  cor.test(Education, Fertility)
```

Pearson's product-moment correlation

data: Education and Fertility

t = -5.9536, df = 45, p-value = 3.659e-07

alternative hypothesis: true correlation is not equal to 0

95 percent confidence interval:

-0.7987075 -0.4653206

sample estimates:

cor

-0.6637889

What is the Pipe?, cont.



Tip

The pipe makes your code read like a **recipe**: take this data → then filter → then group → then summarize. It eliminates deep nesting and temporary variables.

The . Placeholder

When the piped data is **not** the first argument, use `.` as a placeholder:

```
swiss %>% head(., n = 4)
```

	Fertility	Agriculture	Examination	Education	Catholic
Courtelary	80.2	17.0	15	12	9.96
Delemont	83.1	45.1	6	9	84.84
Franches-Mnt	92.5	39.7	5	5	93.40
Moutier	85.8	36.5	12	7	33.77

Infant.Mortality

Courtelary	22.2
Delemont	22.2
Franches-Mnt	20.2
Moutier	20.3

```
swiss %>% do(head(., n = 4))
```

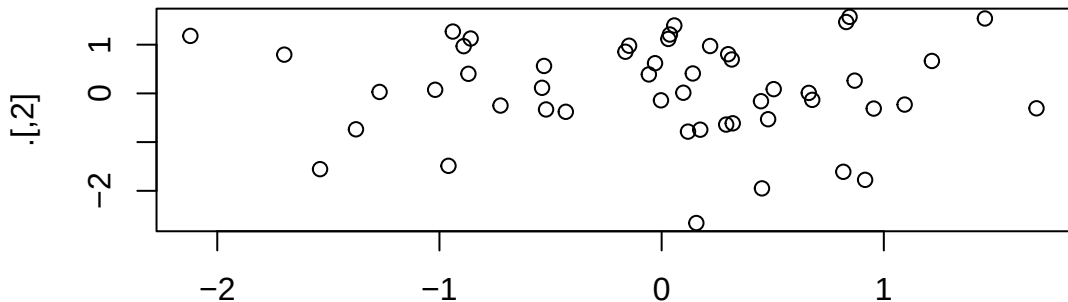
	Fertility	Agriculture	Examination	Education	Catholic
Courtelary	80.2	17.0	15	12	9.96
Delemont	83.1	45.1	6	9	84.84
Franches-Mnt	92.5	39.7	5	5	93.40
Moutier	85.8	36.5	12	7	33.77



The Tee Pipe %T>%

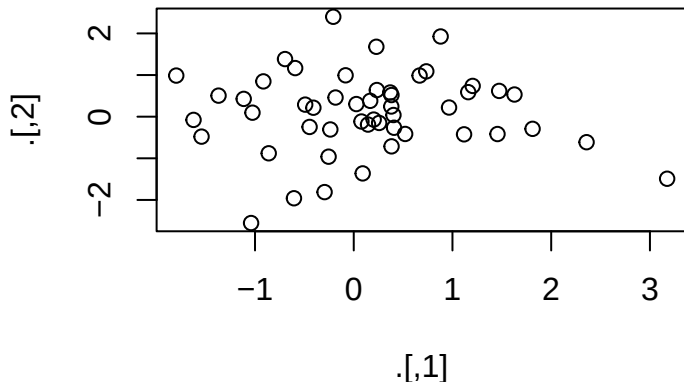
%T>% returns the **left-hand side** (upstream value), not the right-hand side result:

```
# res1 will be NULL — because plot() returns NULL  
res1 <- rnorm(100) %>% matrix(ncol = 2) %>% plot()
```



The Tee Pipe %T>%, cont.

```
# res2 keeps the matrix — %T>% returns the left side  
res2 <- rnorm(100) %>% matrix(ncol = 2) %T>% plot()
```



The Exposition Pipe %\$%

%\$% exposes column names — like `attach()` but safer:

```
# Instead of:
with(mtcars, cor.test(cyl, mpg))
```

Pearson's product-moment correlation

```
data:  cyl and mpg
t = -8.9197, df = 30, p-value = 6.113e-10
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.9257694 -0.7163171
sample estimates:
      cor
-0.852162
```



The Exposition Pipe %\$%, cont.

```
# Use exposition pipe:  
mtcars %$% cor.test(cyl, mpg)
```

Pearson's product-moment correlation

```
data:  cyl and mpg  
t = -8.9197, df = 30, p-value = 6.113e-10  
alternative hypothesis: true correlation is not equal to 0  
95 percent confidence interval:  
  -0.9257694 -0.7163171  
sample estimates:  
      cor  
-0.852162
```

The Assignment Pipe %<>%

%<>% pipes and assigns back to the original variable:

```
mtcars_small <- mtcars %>% select(mpg, cyl, disp) %>% head(3)
mtcars_small
```

	mpg	cyl	disp
Mazda RX4	21.0	6	160
Mazda RX4 Wag	21.0	6	160
Datsun 710	22.8	4	108

```
# %<>% modifies in place
mtcars_small %<>% transform(cyl = cyl * 2)
mtcars_small
```

	mpg	cyl	disp
Mazda RX4	21.0	12	160
Mazda RX4 Wag	21.0	12	160
Datsun 710	22.8	8	108

Base R Pipe |>

Since **R 4.1.0**, base R includes its own pipe `|>`. It works similarly but has some differences (e.g., no `.` placeholder for non-first-argument). This course uses `%>%` from `magrittr`.

See: <https://www.tidyverse.org/blog/2023/04/base-vs-magrittr-pipe/>

Section 2

dplyr Core Verbs

What is dplyr?

dplyr is the data manipulation grammar of the tidyverse:

- Works on data frames and tibbles
- **Row-based** operations (fast, consistent API)
- Five core verbs: `select()`, `filter()`, `mutate()`, `summarise()`, `arrange()`
- Designed to work seamlessly with `%>%`
- dplyr official page
- R for Data Science



Figure 1: dplyr logo

dplyr Core Verbs Overview

Verb	What it does	Row change?
<code>select()</code>	Choose columns by name	No
<code>filter()</code>	Subset rows by condition	Fewer
<code>mutate()</code>	Create/modify columns	No
<code>summarise()</code>	Aggregate to single values	Fewer
<code>arrange()</code>	Sort rows	No

All verbs work naturally with `group_by()` for grouped operations.

select() — Choose Columns

Select columns by name, position, or helper functions:

```
# Get ready
library(tidyverse)

# Basic selection
starwars %>% select(name, height, mass) %>% head(3)
```

```
# A tibble: 3 x 3
  name          height mass
  <chr>         <int> <dbl>
1 Luke Skywalker    172     77
2 C-3P0             167     75
3 R2-D2              96     32
```

select() — Choose Columns, cont.

```
# Range of columns (name through mass)
starwars %>% select(name:mass) %>% head(3)
```

```
# A tibble: 3 x 3
  name          height mass
  <chr>         <int> <dbl>
1 Luke Skywalker   172    77
2 C-3PO           167    75
3 R2-D2            96    32
```

```
# Helper functions
starwars %>% select(name, ends_with("color")) %>% head(3)
```

```
# A tibble: 3 x 4
  name          hair_color skin_color eye_color
  <chr>         <chr>      <chr>      <chr>
1 Luke Skywalker blond      fair       blue
2 C-3PO         <NA>      gold       yellow
3 R2-D2         <NA>      white, blue red
```

select() Helpers

Helper functions for select - ?select

select(iris, contains(""))

Select columns whose name contains a character string.

select(iris, ends_with("Length"))

Select columns whose name ends with a character string.

select(iris, everything())

Select every column.

select(iris, matches(".t."))

Select columns whose name matches a regular expression.

select(iris, num_range("x", 1:5))

Select columns named x1, x2, x3, x4, x5.

select(iris, one_of(c("Species", "Genus")))

Select columns whose names are in a group of names.

select(iris, starts_with("Sepal"))

Select columns whose name starts with a character string.

select(iris, Sepal.Length:Petal.Width)

Select all columns between Sepal.Length and Petal.Width (inclusive).

select(iris, -Species)

Select all columns except Species.

Figure 2: dplyr::select helpers

select() Helpers, cont.

Common helpers:

- `starts_with("prefix")`
- `ends_with("suffix")`
- `contains("substring")`
- `matches("regex")`
- `everything()` — all remaining columns
- `last_col()` — the last column
- Remove columns with `-`: `select(-hair_color, -eye_color)`

filter() — Subset Rows

Keep rows that satisfy logical conditions:

```
# Get mouse gene data
mouse.tibble <- read_delim(
  file = "data/talk04/mouse_genes_biomart_sep2018.txt",
  delim = "\t", quote = ""
)
```

filter() — Subset Rows, cont.

```
# Filter for specific chromosomes
mouse.tibble %>%
  filter(`Chromosome/scaffold name` %in% c(1:19, "X", "Y")) %>%
  count(`Chromosome/scaffold name`) %>%
  arrange(desc(n))
```

```
# A tibble: 21 x 2
  `Chromosome/scaffold name`      n
  <chr>                <int>
1 7                    12344
2 2                    10877
3 5                     8955
4 11                   8673
5 1                     8553
6 9                     8030
7 6                     7845
8 4                     7573
9 3                     6938
10 10                   6568
# i 11 more rows
```

filter() — Multiple Conditions

Combine conditions with & (AND), | (OR), , (AND):

```
# Blond AND blue-eyed characters
starwars %>%
  select(name, ends_with("color"), gender, species) %>%
  filter(hair_color == "blond", eye_color == "blue")
```

A tibble: 3 x 6

	name	hair_color	skin_color	eye_color	gender	species
	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	Luke Skywalker	blond	fair	blue	masculine	Human
2	Anakin Skywalker	blond	fair	blue	masculine	Human
3	Finis Valorum	blond	fair	blue	masculine	Human

filter() — Multiple Conditions, cont.

```
# Filter for specific gene types
mouse.tibble %>%
  filter(`Transcript type` == "protein_coding" |
         `Transcript type` == "miRNA" | `Transcript type` == "lincRNA") %>%
  count(`Transcript type`) %>%
  arrange(desc(n))
```

```
# A tibble: 3 x 2
  `Transcript type`      n
  <chr>                <int>
1 protein_coding      58384
2 lincRNA              8557
3 miRNA                2265
```


filter() — Multiple Conditions, cont.

the above | operations can be easily replaced with %in%:

```
mouse.tibble %>%
  filter(`Transcript type` %in% c("protein_coding", "miRNA", "lincRNA")) %>%
  count(`Transcript type`) %>%
  arrange(desc(n))
```

```
# A tibble: 3 x 2
  `Transcript type`      n
  <chr>                <int>
1 protein_coding      58384
2 lincRNA              8557
3 miRNA               2265
```

mutate() — Create/Modify Columns

Add new columns computed from existing ones (does **not** change row count):

```
# Calculate BMI for starwars characters
starwars %>%
  select(name, height, mass) %>%
  mutate(bmi = mass / ((height / 100)^2)) %>%
  head(6)
```

```
# A tibble: 6 x 4
  name          height  mass   bmi
  <chr>         <int> <dbl> <dbl>
1 Luke Skywalker   172    77  26.0
2 C-3PO            167    75  26.9
3 R2-D2            96     32  34.7
4 Darth Vader      202   136  33.3
5 Leia Organa      150    49  21.8
6 Owen Lars        178   120  37.9
```

mutate() — Create/Modify Columns, cont.

```
# Create a categorized column
starwars %>%
  select(name, species, height) %>%
  mutate(
    height_cat = case_when(
      height < 100 ~ "Short",
      height < 180 ~ "Medium",
      height >= 180 ~ "Tall"
    )
  ) %>%
  head(6)
```

```
# A tibble: 6 x 4
  name          species height height_cat
<chr>          <chr>   <int> <chr>
1 Luke Skywalker Human     172 Medium
2 C-3PO         Droid     167 Medium
3 R2-D2         Droid      96 Short
4 Darth Vader   Human     202 Tall
5 Leia Organa   Human     150 Medium
6 Owen Lars     Human     178 Medium
```



arrange() — Sort Rows

Sort by one or more columns; use `desc()` for descending:

```
# Sort starwars by height (tallest first)
starwars %>%
  select(name, height, mass) %>%
  arrange(desc(height)) %>%
  head(6)
```

```
# A tibble: 6 x 3
  name      height  mass
  <chr>      <int> <dbl>
1 Yarael Poof    264    NA
2 Tarfful       234   136
3 Lama Su       229    88
4 Chewbacca     228   112
5 Roos Tarpals  224    82
6 Grievous      216   159
```



arrange() — Sort Rows, cont.

```
# Sort by species then by height
starwars %>%
  select(name, species, height) %>%
  arrange(species, desc(height)) %>%
  head(10)
```

```
# A tibble: 10 x 3
```

	name	species	height
	<chr>	<chr>	<int>
1	Ratts Tyerel	Aleena	79
2	Dexter Jettster	Besalisk	198
3	Ki-Adi-Mundi	Cerean	198
4	Mas Amedda	Chagrian	196
5	Zam Wesell	Clawdite	168
6	IG-88	Droid	200
7	C-3PO	Droid	167
8	R5-D4	Droid	97
9	R2-D2	Droid	96
10	R4-P17	Droid	96

group_by() + summarise() — Grouped Aggregation

The most powerful combination in dplyr:

```
# Average height and mass by species
starwars %>%
  group_by(species) %>%
  summarise(
    n = n(),
    avg_height = mean(height, na.rm = TRUE),
    avg_mass = mean(mass, na.rm = TRUE)
  ) %>%
  arrange(desc(n))
```

A tibble: 38 x 4

	species	n	avg_height	avg_mass
	<chr>	<int>	<dbl>	<dbl>
1	Human	35	178	81.3
2	Droid	6	131.	69.8
3	<NA>	4	175	81
4	Gungan	3	209.	74
5	Kaminoan	2	221	88
6	Mirialan	2	168	53.1
7	Twi'lek	2	170	55



Full Analysis Example — Mouse Genes

```

dat <- mouse.tibble %>%
  # 1. Keep only main chromosomes
  filter(`Chromosome/scaffold name` %in% c(1:19, "X", "Y")) %>%
  # 2. Keep only three gene types
  filter(`Transcript type` %in% c("protein_coding", "miRNA", "lincRNA")) %>%
  # 3. Rename columns for readability
  select(
    CHR = `Chromosome/scaffold name`,
    TYPE = `Transcript type`,
    GENE_ID = `Gene stable ID`,
    GENE_LEN = `Transcript length (including UTRs and CDS)`
  ) %>%
  # 4. Group and summarize
  group_by(CHR, TYPE) %>%
  summarise(
    count = n_distinct(GENE_ID),
    mean_len = mean(GENE_LEN)
  ) %>%
  # 5. Sort
  arrange(CHR, desc(count))

```

Full Analysis — Result

CHR	TYPE	count	mean_len
1	protein_coding	1200	2699.59009
1	lincRNA	347	1206.76149
1	miRNA	128	97.97656
10	protein_coding	1020	2408.16454
10	lincRNA	398	1220.35543
10	miRNA	91	89.87912
11	protein_coding	1640	2431.87666
11	lincRNA	189	1134.49174
11	miRNA	137	87.48905
12	protein_coding	644	2523.94822
12	lincRNA	327	1277.14979
12	miRNA	146	86.24658
13	protein_coding	831	2380.41499
13	lincRNA	428	1251.04552
13	miRNA	97	105.52577

Grades Analysis Example

A complete analysis with `pivot_longer` + `dplyr`:

```
# Create grades tibble
set.seed(42)
grades <- tibble(
  "Name" = c("Weihua Chen", "Mm Hu", "John Doe", "Jane Doe",
            "Warren Buffet", "Elon Musk", "Jack Ma"),
  "Occupation" = c("Teacher", "Student", "Teacher", "Student",
                  rep("Entrepreneur", 3)),
  "English" = sample(60:100, 7),
  "ComputerScience" = sample(80:90, 7),
  "Biology" = sample(50:100, 7),
  "Bioinformatics" = sample(40:90, 7)
)
```

Grades Analysis Example, cont.

```
kbl(grades, booktabs = TRUE)
```

Name	Occupation	English	ComputerScience	Biology	Bioinformatics
Weihua Chen	Teacher	96	86	99	76
Mm Hu	Student	60	83	96	70
John Doe	Teacher	84	88	52	84
Jane Doe	Student	69	84	90	44
Warren Buffet	Entrepreneur	95	85	74	59
Elon Musk	Entrepreneur	77	89	76	73
Jack Ma	Entrepreneur	83	81	85	67

Grades — Average per Student

```
grades %>%
  pivot_longer(cols = c(English, ComputerScience, Biology, Bioinformatics),
               names_to = "course", values_to = "grade") %>%
  group_by(Name, Occupation) %>%
  summarise(
    avg_grade = mean(grade),
    courses_count = n()
  ) %>%
  arrange(desc(avg_grade)) %>%
  kbl(booktabs = TRUE)
```

Name	Occupation	avg_grade	courses_count
Weihua Chen	Teacher	89.25	4
Jack Ma	Entrepreneur	79.00	4
Elon Musk	Entrepreneur	78.75	4
Warren Buffet	Entrepreneur	78.25	4
Mm Hu	Student	77.25	4
John Doe	Teacher	77.00	4
Jane Doe	Student	71.75	4



Grades — Best Subject per Student

```
# Find each student's best subject
grades %>%
  pivot_longer(cols = c(English, ComputerScience, Biology, Bioinformatics),
               names_to = "course", values_to = "grade") %>%
  group_by(Name) %>%
  summarise(
    best_course = first(course, order_by = desc(grade)),
    best_grade = first(grade, order_by = desc(grade)),
    avg_grade = mean(grade)
  ) %>%
  arrange(desc(avg_grade)) %>%
  kbl(booktabs = TRUE)
```

Name	best_course	best_grade	avg_grade
Wei-hua Chen	Biology	99	89.25
Jack Ma	Biology	85	79.00
Elon Musk	ComputerScience	89	78.75
Warren Buffet	English	95	78.25
Mm Hu	Biology	96	77.25

Section 3

tidyr — Reshaping Data

Why Reshape Data?

Data comes in two forms:

Format	Description	Example
Wide	Each variable has its own column; good for humans	One column per gene
Long	Each row is one observation; good for computers	One row per gene-sample pair

tidyr provides tools to switch between these formats.

pivot_longer() — Wide → Long

Convert columns into rows:

```
# Wide gene expression data
wide_expr <- tibble(
  patient = c("P1", "P2", "P3"),
  BRCA1 = c(2.3, 1.8, 3.1),
  TP53 = c(-1.2, -0.8, 0.5),
  EGFR = c(0.5, 1.1, -0.3)
)
kbl(wide_expr, booktabs = TRUE)
```

patient	BRCA1	TP53	EGFR
P1	2.3	-1.2	0.5
P2	1.8	-0.8	1.1
P3	3.1	0.5	-0.3

pivot_longer() — Wide → Long, cont.

```
# Convert to long format
wide_expr %>%
  pivot_longer(
    cols = c(BRCA1, TP53, EGFR),
    names_to = "gene",
    values_to = "expression"
  ) %>%
  kbl(booktabs = TRUE)
```

patient	gene	expression
P1	BRCA1	2.3
P1	TP53	-1.2
P1	EGFR	0.5
P2	BRCA1	1.8
P2	TP53	-0.8
P2	EGFR	1.1
P3	BRCA1	3.1
P3	TP53	0.5
P3	EGFR	-0.3



pivot_longer() Explained

```
# Anatomy of pivot_longer()
wide_expr %>%
  pivot_longer(
    cols      = c(BRCA1, TP53, EGFR), # which columns to pivot
    names_to  = "gene",                # new column for old column names
    values_to = "expression"          # new column for values
  )
```

pivot_longer() Explained, cont.

Take the grades2 data as an example:

```
# Load grades2 data
grades2 <- read_tsv(file = "data/talk06/grades2.txt")
grades2
```

A tibble: 3 x 6

	name	Microbiology	English	Chinese	Bioinformatics	Chemistry
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Zhi Liu	100	50	69	NA	NA
2	Weihua Chen	89	99	NA	99	NA
3	Kang Ning	NA	NA	20	100	76

- `cols = -name` — pivot all columns except name (the ID column)
- `names_to = "course"` — new column for old column names (Microbiology, English, Chinese, etc.)
- `values_to = "grade"` — new column for the scores
- `values_drop_na = TRUE` removes rows with missing grades

pivot_longer() — Visual Guide

-name: 此列保留

列名变为第一列，取名为 **course**

name	Bioinformatics	Chemistry	Chinese	English	Microbiology
Kang Ning	100	76	20	NA	NA
Wei-hua Chen	99	NA	NA	99	89
Zhi Liu	NA	NA	69	50	100

值变为第二列，取名为 **grade**

Figure 3: pivot_longer() explained

This figure illustrates the wide-to-long transformation: column names become values in a “key” column, and cell values go into a “value” column.



pivot_wider() — Long → Wide

Reverse the operation — convert rows back to columns:

```
long_data <- wide_expr %>%
  pivot_longer(cols = c(BRCA1, TP53, EGFR),
               names_to = "gene", values_to = "expression")

# Convert back to wide
long_data %>%
  pivot_wider(
    names_from = gene,
    values_from = expression
  ) %>%
  kbl(booktabs = TRUE)
```

patient	BRCA1	TP53	EGFR
P1	2.3	-1.2	0.5
P2	1.8	-0.8	1.1
P3	3.1	0.5	-0.3

pivot_wider() Explained

Using the grades2 data (from the previous pivot_longer result):

```
# Long format: each row is one student × one course
grades2_long <- grades2 %>%
  pivot_longer(-name, names_to = "course", values_to = "grade",
               values_drop_na = TRUE)
grades2_long
```

```
# A tibble: 9 × 3
  name      course      grade
  <chr>    <chr>    <dbl>
1 Zhi Liu  Microbiology  100
2 Zhi Liu  English      50
3 Zhi Liu  Chinese      69
4 Weihua Chen Microbiology  89
5 Weihua Chen English    99
6 Weihua Chen Bioinformatics 99
7 Kang Ning  Chinese     20
8 Kang Ning  Bioinformatics 100
9 Kang Ning  Chemistry   76
```

pivot_wider() — Visual Guide

1. 选取哪列做“列名”

2. 选取哪列做“数据”

name	course	grade
Zhi Liu	Microbiology	100
Zhi Liu	English	50
Zhi Liu	Chinese	69
Weihua Chen	Microbiology	89
Weihua Chen	English	99
Weihua Chen	Bioinformatics	99
Kang Ning	Bioinformatics	100
Kang Ning	Chinese	20
Kang Ning	Chemistry	76

Figure 4: pivot_wider() explained

- `names_from = course` — which column provides the **new column names** (Microbiology, English, Chinese, etc.)
- `values_from = grade` — which column provides the **values** to fill into the new columns
- The result reconstructs the original wide table with one row per student

separate() — Split One Column into Many

Split a column based on a separator:

```
# Data with combined column
drug_data <- tibble(
  treatment = c("DrugA_10mg", "DrugA_20mg", "DrugB_10mg", "DrugB_20mg"),
  response = c(0.8, 0.6, 0.3, 0.1)
)
kbl(drug_data, booktabs = TRUE)
```

treatment	response
DrugA_10mg	0.8
DrugA_20mg	0.6
DrugB_10mg	0.3
DrugB_20mg	0.1

separate() — Split One Column into Many, cont.

```
drug_data %>%  
  separate(treatment, into = c("drug", "dose"), sep = "_") %>%  
  kbl(booktabs = TRUE)
```

drug	dose	response
DrugA	10mg	0.8
DrugA	20mg	0.6
DrugB	10mg	0.3
DrugB	20mg	0.1

unite() — Combine Columns into One

The reverse of `separate()`:

```
drug_data %>%
  separate(treatment, into = c("drug", "dose"), sep = "_") %>%
  unite(col = "treatment_label", drug, dose, sep = " - ") %>%
  kbl(booktabs = TRUE)
```

treatment_label	response
DrugA - 10mg	0.8
DrugA - 20mg	0.6
DrugB - 10mg	0.3
DrugB - 20mg	0.1

Section 4

Joining Tables

Why Join Tables?

Real-world data is rarely in a single table. Common scenarios:

- **Clinical data:** patient info + lab results + treatment records
- **Genomics:** gene expression + gene annotation + pathway data
- **Microbiome:** OTU table + taxonomy + sample metadata

dplyr provides a family of join functions that work like SQL joins.

Join Types Overview

Join	Keeps rows from	Description
<code>left_join(x, y)</code>	All x	Match y to x; NA where no match
<code>right_join(x, y)</code>	All y	Match x to y; NA where no match
<code>inner_join(x, y)</code>	Both	Only rows with matches in both
<code>full_join(x, y)</code>	All	All rows from both; NA where no match
<code>semi_join(x, y)</code>	x	Keep x rows that have a match in y
<code>anti_join(x, y)</code>	x	Keep x rows that have no match in y

Join Example

```
# Patient metadata
patients <- tibble(
  id = c("P01", "P02", "P03", "P04"),
  age = c(45, 62, 38, 55),
  sex = c("F", "M", "F", "M")
)

# Lab results
lab_results <- tibble(
  id = c("P01", "P02", "P03", "P05"),
  glucose = c(98, 142, 105, 130),
  cholesterol = c(180, 220, 195, 210)
)
```

Join Example, Inner Join

```
# Inner join — only patients with lab results
patients %>% inner_join(lab_results, by = "id") %>%
  kbl(booktabs = TRUE)
```

id	age	sex	glucose	cholesterol
P01	45	F	98	180
P02	62	M	142	220
P03	38	F	105	195

Left Join

```
# Left join — all patients, NA if no lab result
patients %>% left_join(lab_results, by = "id") %>%
  kbl(booktabs = TRUE)
```

id	age	sex	glucose	cholesterol
P01	45	F	98	180
P02	62	M	142	220
P03	38	F	105	195
P04	55	M	NA	NA

Full Join

```
# Full join — all rows from both tables
patients %>% full_join(lab_results, by = "id") %>%
  kbl(booktabs = TRUE)
```

id	age	sex	glucose	cholesterol
P01	45	F	98	180
P02	62	M	142	220
P03	38	F	105	195
P04	55	M	NA	NA
P05	NA	NA	130	210

Semi Join

```
# Semi join — only patients with lab results
patients %>% semi_join(lab_results, by = "id") %>%
  kbl(booktabs = TRUE)
```

id	age	sex
P01	45	F
P02	62	M
P03	38	F

Note: `semi_join()` keeps only the rows in `x` that have a match in `y`, but doesn't bring over any information from `y`. This is different from `inner_join()` which brings over all columns from `y`.

Anti Join — Find Missing Matches

Useful for quality control — find which rows have **no** match:

```
# Which patients have NO lab results?
patients %>% anti_join(lab_results, by = "id") %>%
  kbl(booktabs = TRUE)
```

id	age	sex
P04	55	M

```
# Which lab results have NO matching patient?
lab_results %>% anti_join(patients, by = "id") %>%
  kbl(booktabs = TRUE)
```

id	glucose	cholesterol
P05	130	210

Section 5

Advanced dplyr

across() — Column-wise Operations

Apply a function to multiple columns:

```
# Summarize all numeric columns
starwars %>%
  summarise(
    across(where(is.numeric), ~ mean(.x, na.rm = TRUE))
  ) %>%
  kbl(booktabs = TRUE)
```

height	mass	birth_year
174.6049	97.31186	87.56512

across() — Column-wise Operations, cont.

```
# Mutate multiple columns
starwars %>%
  select(name, height, mass) %>%
  mutate(
    across(c(height, mass), ~ .x / 100)
  ) %>%
  head(4) %>%
  kbl(booktabs = TRUE)
```

name	height	mass
Luke Skywalker	1.72	0.77
C-3PO	1.67	0.75
R2-D2	0.96	0.32
Darth Vader	2.02	1.36

rowwise() — Row-wise Operations

By default, dplyr operations are column-wise. `rowwise()` makes them row-wise:

```
# Row-wise mean of gene expression
gene_expr <- tibble(
  gene = c("BRCA1", "TP53", "EGFR"),
  sample1 = c(2.3, 1.8, 0.5),
  sample2 = c(2.1, 1.5, 0.8),
  sample3 = c(2.5, 2.0, 0.3)
)

gene_expr %>%
  rowwise() %>%
  mutate(mean_expr = mean(c_across(starts_with("sample")))) %>%
  kbl(booktabs = TRUE)
```

gene	sample1	sample2	sample3	mean_expr
BRCA1	2.3	2.1	2.5	2.3000000
TP53	1.8	1.5	2.0	1.7666667
EGFR	0.5	0.8	0.3	0.5333333



slice_*() — Row Selection by Position

Select rows by position within groups:

```
# Top 3 tallest characters per species (species with 3 members)
```

```
starwars %>%
  filter(!is.na(height)) %>%
  group_by(species) %>%
  filter(n() >= 3) %>%
  slice_max(order_by = height, n = 3) %>%
  select(name, species, height) %>%
  head(9) %>%
  kbl(booktabs = TRUE)
```

name	species	height
IG-88	Droid	200
C-3PO	Droid	167
R5-D4	Droid	97
Roos Tarpals	Gungan	224
Rugor Nass	Gungan	206
Jar Jar Binks	Gungan	196



count() — Quick Frequency Table

A convenient shortcut for `group_by()` + `summarise(n = n())`:

```
# Count by species, sorted by frequency
starwars %>%
  count(species, sort = TRUE) %>%
  head(8) %>%
  kbl(booktabs = TRUE)
```

species	n
Human	35
Droid	6
NA	4
Gungan	3
Kaminoan	2
Mirialan	2
Twi'lek	2
Wookiee	2

count() — Quick Frequency Table, cont.

```
# Count with weight (e.g., total mass per species)
starwars %>%
  count(species, wt = mass, sort = TRUE) %>%
  head(5) %>%
  kbl(booktabs = TRUE)
```

species	n
Human	1626.2
Hutt	1358.0
Droid	279.0
Wookiee	248.0
NA	243.0

Section 6

Summary & Exercises

What We Covered Today

The Pipe %>%

- Basic usage, . placeholder
- %T>%, %\$%, %<>%
- Base R pipe |>

dplyr Core Verbs

- select() — choose columns
- filter() — subset rows
- mutate() — create/modify columns
- summarise() + group_by()
- arrange() — sort rows

tidyr Reshaping

- pivot_longer() — wide → long
- pivot_wider() — long → wide
- separate() / unite()

Joining Tables

- left_join, inner_join, full_join
- semi_join, anti_join

Advanced

- across() — column-wise ops
- rowwise() — row-wise ops
- slice_*(), count()

Next Time

- **Talk 05:** Data Visualization with **ggplot2** — grammar of graphics, scatter plots, bar charts, boxplots, facets, themes

Exercises

- Homework_AI_powered_R directory → talk04-homework.qmd
- Covers: pipe, dplyr verbs, tidyr reshaping, joins
- Due: check DingTalk group for deadline

All code available at: <https://github.com/evolgeniusteam/R-for-bioinformatics>